Streamlit Part 2: IEM 4723 Information Systems Design

Dr. Paritosh Ramanan

School of Industrial Engineering and Management
Oklahoma State University

Fall 2026

Adapted from the Streamlit documentation, https://docs.streamlit.io, and 30 Days of Streamlit, https://30days.streamlit.app.

1. Input Widgets
2. Layout
3. State Across Reruns
4. Talking to MySQL and Redis

Input Widgets

The Widget Catalogue

Widget	Returns
`st.button`	`True` on the run where it was clicked
`st.checkbox`, `st.toggle`	a boolean
`st.radio`, `st.selectbox`	the one option chosen
`st.multiselect`	a list of the options chosen
`st.slider`, `st.number_input`	a number
`st.select_slider`	the item chosen from a list
`st.text_input`, `st.text_area`	a string
`st.date_input`, `st.time_input`	a date or time object
`st.file_uploader`	the uploaded file object

- Every one of them draws itself and returns the user's current choice, which is the pattern from last lecture.
- The first argument is always the label the user sees. Give it a real one; it is also what a screen reader announces.

Source: Streamlit documentation, API reference, Input widgets

Reading What the User Chose

```
carrier = st.selectbox("Carrier",
                       ["DHL Ground", "FedEx", "UPS"])
states = st.multiselect("State", ["PA", "OK", "TX"])
min_qty = st.slider("Minimum quantity", 0, 50, 5)
show_all = st.checkbox("Include shipped orders")
```

- After these four lines, `carrier` is a string, `states` is a list, `min_qty` is an integer and `show_all` is a boolean. From here on it is ordinary Python.
- The third argument to `st.slider` is the default. Widgets generally take their default as the argument after the range or the options.
- Nothing has been filtered yet. The widgets only collect the choices; the next slide uses them.

A Filter, End to End

```
@st.cache_data
def load_orders():
    return pd.read_csv("data/orders/CustomerOrder.csv")

orders = load_orders()

carrier = st.selectbox("Carrier",
                       sorted(orders["Order_Carrier"].unique()))

filtered = orders[orders["Order_Carrier"] == carrier]

st.metric("Orders with this carrier", len(filtered))
st.dataframe(filtered)
```

- This is a complete, useful application. Change the dropdown, the script reruns, `carrier` holds the new value, and the table and the count both update.
- The `@st.cache_data` matters here. Without it the CSV is read again on every change of the dropdown.
- Note that the options came from the data rather than being typed out, so the app cannot offer a carrier that does not exist.

Layout

The Sidebar

```
with st.sidebar:
    carrier = st.selectbox("Carrier", carriers)
    min_qty = st.slider("Minimum quantity", 0, 50, 5)

st.sidebar.button("Refresh") # the same thing, one line
```

- `st.sidebar` puts elements in a panel on the left that stays put while the main area scrolls.
- The convention, and it is a good one, is that controls go in the sidebar and results go in the main area. A user then knows where to look to change something and where to look for the answer.
- Anything that works in the main area works in the sidebar, either with the `with` block or the `st.sidebar.` prefix.

Columns and Tabs

```
left, right = st.columns(2)
left.metric("Open orders", 128)
right.metric("Late shipments", 7)

tab1, tab2 = st.tabs(["Chart", "Raw data"])
with tab1:
    st.line_chart(daily)
with tab2:
    st.dataframe(orders)
```

- `st.columns` returns one container per column, and they can be given different widths with `st.columns([2, 1])`.
- `st.tabs` returns one container per tab. Use tabs when the content is alternative views of the same thing, and columns when the content belongs side by side.

Expanders and Containers

```
with st.expander("Show the SQL behind this"):
    st.code(query, language="sql")

box = st.container()
box.write("Written into the container")

slot = st.empty()
slot.write("this will be replaced")
```

- `st.expander` hides detail behind a collapsed heading. It is the right home for the query, the assumptions, and anything a curious user may want but most will not.
- `st.container` groups elements so they can be filled later in the script than where they appear on the page.
- `st.empty` holds exactly one element and replaces it when written again, which is how a value gets updated in place.

Source: Streamlit documentation, API reference, Layouts and containers

State Across Reruns

The Problem With Reruns

```
count = 0

if st.button("Add one"):
    count += 1

st.write("Count:", count) # always 0 or 1, never 2
```

- Click the button repeatedly and the count never passes one. This is not a bug in the library; it follows from the model in the last lecture.
- Every click reruns the script from the top, and the first line sets `count` back to zero before the button is ever examined.
- An ordinary Python variable cannot survive a rerun, because the statement that created it runs again.

Session State

```
if "count" not in st.session_state:
    st.session_state.count = 0

if st.button("Add one"):
    st.session_state.count += 1

st.write("Count:", st.session_state.count)
```

- `st.session_state` is a dictionary that survives reruns. Both `st.session_state.count` and `st.session_state["count"]` work.
- The guarded initialisation is the standard pattern and is not optional. Reading a key that was never set raises an exception.
- It is *session* state. Each browser session has its own, so two users of the same app do not share it, which is the opposite of what `@st.cache_resource` does.

Source: Streamlit documentation, Add statefulness to apps

Widget Keys

```
st.slider("Temperature", -100, 100, key="celsius")
st.write(st.session_state.celsius)
```

- Giving a widget a `key` puts its value into session state under that name automatically. The widget and `st.session_state.celsius` are then two views of the same thing.
- This is how one part of a script reads a widget that is created somewhere else entirely, without passing the return value around.
- Keys must be unique across the app. Two widgets sharing a key is an error, and it is the usual cause of a duplicate key message.

Callbacks

```
def increment():
    st.session_state.count += 1

st.button("Add one", on_click=increment)
st.button("Add five", on_click=increment_by, args=(5,))
```

- Most widgets accept `on_change`, and buttons accept `on_click`. The function runs *before* the script reruns, so the rest of the script sees the updated state.
- Extra arguments go in `args` as a tuple, or `kwargs` as a dictionary.
- Callbacks are not required for most apps. Reach for one when something must happen exactly once per interaction, rather than every time the script runs.

Lecture 4, Re-read

```
def submit_name():
    st.session_state.submitted = True

st.text_input("Please enter your name:",
              key="name", on_change=submit_name)

if st.session_state.get("submitted", False):
    st.write(f"My name is: {st.session_state.name}")
```

- `key="name"` puts whatever is typed into `st.session_state.name`.
- `on_change=submit_name` runs when the user commits the text, setting a flag that survives later reruns. Without it the greeting would appear and vanish.
- `.get("submitted", False)` is the defensive read: on the very first run the key does not exist yet, and a plain attribute access would raise.

Source: `docker_tutorial/networking/app/main.py`

Forms

```
with st.form("order_filter"):
    carrier = st.selectbox("Carrier", carriers)
    min_qty = st.slider("Minimum quantity", 0, 50, 5)
    submitted = st.form_submit_button("Apply")

if submitted:
    st.dataframe(run_query(carrier, min_qty))
```

- Inside a form, changing a widget does *not* rerun the script. Nothing happens until the submit button is pressed, and then everything is read at once.
- That matters when the work behind the page is expensive. Without a form, a three widget filter can fire three separate database queries while the user is still making up their mind.
- Every form needs exactly one `st.form_submit_button`.

MySQL and Redis

Talking to MySQL and Redis

```
@st.cache_resource
def get_mysql():
    return pymysql.connect(host="mysql", port=3306,
                           user="root", password="OSUGoPokes",
                           database="iem4723_mysql_tutorial")

@st.cache_data(ttl=300)
def orders_by_carrier():
    cur = get_mysql().cursor()
    cur.execute("SELECT Order_Carrier, COUNT(*) "
                "FROM customerorder GROUP BY Order_Carrier")
    return pd.DataFrame(cur.fetchall(),
                        columns=["Carrier", "Orders"])
```

- The connection is a *resource*, opened once and shared. The query result is *data*, copied per caller and given a five minute lifetime.
- Getting these two the wrong way round is the common mistake. A connection under `cache_data` would be copied and quickly break; a dataframe under `cache_resource` is shared mutable state between users.
- `host="mysql"` is the compose service name, exactly as in the notebooks. The app container has to be on the same compose network to resolve it.

What We Have Not Covered

- **Multipage apps.** `st.Page` and `st.navigation` turn several scripts into one app with a navigation menu.
- **Theming and configuration.** Colours and fonts are set in a `[theme]` section of the config file rather than in the script.
- **Secrets.** `st.secrets` keeps passwords out of the source, which the hard-coded password above plainly needs.
- **Fragments and partial reruns.** `st.fragment` reruns one part of a page instead of the whole script, which matters for live and streaming data.
- **Custom components.** The escape hatch for anything the library does not provide.
- All of it is in the documentation at `https://docs.streamlit.io`, and 30 Days of Streamlit works through one command per day.